

错题本

WA 排查手册

面向算法竞赛与 OJ 提交的打印版

整理方式：一般 WA 错误 / 各知识点 WA 错误 / 读题错误
排版目标：A4 打印、目录清晰、代码可读

2026 年 6 月 20 日

目录

第一章 一般 WA 错误	4
1.1 总排查清单	4
1.2 宏定义与数据溢出	4
1.2.1 位运算毒药	4
1.2.2 极大值加法溢出	4
1.2.3 乘法终极溢出	5
1.2.4 浮点数精度丢失	5
1.3 多测并发症	5
1.3.1 中途 return 导致串流	5
1.3.2 清理不干净	5
1.3.3 变量重名覆盖	6
1.4 取模与数学基础	6
1.4.1 负数取模	6
1.4.2 除法取模	6
1.4.3 连加与连乘	6
1.5 边界与极端数据	7
1.6 输出格式	7
1.7 玄学但常见的死角	7
1.7.1 贪心假了	7
1.7.2 比较器写坏	7
1.7.3 未初始化	8
1.7.4 哈希被卡	8

1.8	十五分钟还没找到 WA 时	8
第二章	各个知识点的 WA 错误	9
2.1	图论与树	9
2.1.1	建图与空间	9
2.1.2	图的特殊形态	9
2.1.3	DFS 与 BFS	9
2.1.4	最短路	9
2.1.5	拓扑排序	10
2.1.6	LCA 与树形 DP	10
2.2	动态规划	10
2.2.1	状态定义与初始化	10
2.2.2	遍历顺序	11
2.2.3	状态转移遗漏	11
2.2.4	边界与答案位置	11
2.3	数据结构与技巧	11
2.3.1	并查集	11
2.3.2	线段树	11
2.3.3	二分查找	12
2.3.4	双指针与滑动窗口	12
2.3.5	位运算：优先级与集合操作	12
2.4	数学与精度	13
2.4.1	取模	13
2.4.2	组合计数	13
2.4.3	数论	13
2.4.4	浮点精度	13
第三章	读题错误	14
3.1	数值范围没看清	14
3.1.1	变量可以是负数或 0 吗	14
3.2	核心对象定义没看清	14

3.2.1	Permutation 还是 Array	14
3.2.2	图是不是简单图	14
3.2.3	有向还是无向	15
3.3	操作规则读错	15
3.3.1	同时发生还是依次发生	15
3.3.2	代价是单次还是全局	15
3.3.3	Any 还是 All	15
3.4	坐标系与格式	16
3.4.1	行列坐标与笛卡尔坐标	16
3.4.2	矩阵的顺时针与逆时针旋转	16
3.4.3	输入顺序	16
3.4.4	无解输出	17
附录 A	常用 C++ 板子	18

1 一般 WA 错误

1.1 总排查清单

提交前先把这一页过一遍。大多数 WA 并不是算法完全错了，而是数据范围、初始化、多测、输出格式这些细节把正确思路拖下水。

1. 数据范围：有没有溢出？有没有把 `int` 当成万能药？有没有写 `1 << i`？
2. 多组数据：每一组都清空了吗？有没有在 `solve()` 中提前 `return` 导致输入串流？
3. 取模逻辑：减法是否补了 `+ MOD`？除法是否用了逆元？每一步是否及时取模？
4. 边界样例：最小、最大、全相同、无解、不连通、退化链都测过了吗？
5. 输出格式：大小写、下标基准、空格、换行是否完全按题面来？

1.2 宏定义与数据溢出

1.2.1 位运算毒药

代码中出现 `1 << i` 时，常数 1 默认是 32 位。若 $i \geq 31$ ，很容易直接溢出或变成负数。

修正：需要长整型位移时写 `1LL << i`。如果位数可能更大，先确认题目是否需要 `__int128` 或 `bitset`。

1.2.2 极大值加法溢出

如果设 `const long long inf = 2e18;`，在最短路或 DP 中写 `inf + x`，可能直接冲破 `long long` 上限，变成负数。

修正：转移前判断合法性，例如 `if (dist[u] != inf && dist[v] > dist[u] + w)`。DP 里同理：不可达状态不要参与转移。

1.2.3 乘法终极溢出

两个 10^{18} 级别的数相乘，`long long` 必炸。哪怕最后会取模，中间乘积也已经错了。

修正：大数乘法取模时使用 `__int128`、快速乘，或者保证每一步先取模。

1.2.4 浮点数精度丢失

直接用 `double` 比较分式、判断相等、判断完全平方数，容易被精度误差打穿。

修正：优先用整数比较，例如把 $\frac{a}{b} > \frac{c}{d}$ 改成 $a \cdot d > b \cdot c$ ，同时注意负号和溢出。必须用 `double` 时，判等写 `abs(a - b) < 1e-9`。

1.3 多测并发症

1.3.1 中途 return 导致串流

在 `solve()` 中读到非法情况就直接 `return`，但这一组输入还没有读完，后续数据会被当成下一组测试读入。

修正：先把当前测试的输入完整读完，再决定输出；或者设计读取逻辑时保证提前返回不会留下未读数据。

1.3.2 清理不干净

- 图论中的 `head`、`edge`、`cnt` 是否重置？
- `vector<int> G[N]` 是否只清空了当前 $1 \sim n$ ，却留下上一组更大 n 的残边？

- DP、计数数组、visited 数组是否每组都重新初始化？

1.3.3 变量重名覆盖

全局有 `int n, m;`，在 `solve()` 里又写 `int n, m;` `cin >> n >> m;`，导致其他函数读到的全局变量仍然是 0。

修正：需要被 DFS、BFS、DP 函数共享的变量，统一使用全局变量；局部读入时避免重名。

1.4 取模与数学基础

1.4.1 负数取模

减法取模写成 $(a - b) \% \text{MOD}$ 。当 $a < b$ 时，C++ 的结果仍然是负数。

修正：写成 $(a - b + \text{MOD}) \% \text{MOD}$ ；如果 b 本身可能很大，先写 $(a - b \% \text{MOD} + \text{MOD}) \% \text{MOD}$ 。

1.4.2 除法取模

在模意义下， $(a / b) \% \text{MOD}$ 通常是错的。

修正：除法要变成乘逆元： $a * \text{qpow}(b, \text{MOD} - 2) \% \text{MOD}$ ，或者在非质数模数下用扩展欧几里得判断逆元是否存在。

1.4.3 连加与连乘

$(A * B * C) \% \text{MOD}$ 可能在取模前已经溢出。

修正：步步取模： $A * B \% \text{MOD} * C \% \text{MOD}$ 。加法很多时也要避免临时和越界。

1.5 边界与极端数据

- **极小数据**： $n = 0, 1, 2$ 是否测过？区间、树形 DP、图论最容易在这里翻车。
- **极大数据**：全等数列、全为 10^9 、全为负数时逻辑是否仍然成立？
- **退化结构**：图是一条链、菊花图、森林、孤立点、重边、自环时是否能处理？
- **无解情况**：题面是否规定输出 -1、0、空行或 Impossible？

1.6 输出格式

- **大小写**：题目要求 YES/NO、Yes/No，还是其他格式？
- **下标基准**：输出的是 0-based 还是 1-based？是否需要 +1？
- **空格与换行**：行末是否有多余空格？最后一行是否需要换行？
- **多解要求**：任意解、字典序最小、字典序最大，要求完全不同。

1.7 玄学但常见的死角

1.7.1 贪心假了

代码看起来没有 bug，边界也都过了，但贪心策略无法证明。

修正：主动构造反例：问自己“按照我的选择方式，是否会为了眼前收益错过后续更优解？”无法证明时，优先考虑 DP、二分答案、图模型或交换论证。

1.7.2 比较器写坏

```
// 错误：相等时返回 true，会破坏严格弱序
bool cmp(int a, int b) { return a <= b; }

// 正确：必须严格
bool cmp(int a, int b) { return a < b; }
```

1.7.3 未初始化

本地样例全对，提交 WA，常见原因是局部变量和局部数组没有初值。

修正：所有局部变量显式初始化，例如 `int sum = 0; int mx = -inf;`。数组用 `fill`、`memset` 或构造函数清空。

1.7.4 哈希被卡

`unordered_map` 使用固定哈希，可能在 Codeforces 等平台被构造数据卡成 $O(N^2)$ 。

修正：能用 `map` 就用 `map`；必须用哈希时，加入基于时间戳的 custom hash。

1.8 十五分钟还没找到 WA 时

1. 停止盯代码，手造一组包含负数、0、1、乱序的小样例。
2. 在纸上跑一遍自己的算法。
3. 写一个保证正确的暴力版本，复杂度高也没关系。
4. 写随机数据生成器开始对拍。
5. 找出最小错误数据，再回到代码定位第一处不一致。

2 各个知识点的 WA 错误

2.1 图论与树

2.1.1 建图与空间

- 无向图边数：一条边要存正反两次，链式前向星空间至少开 $2M$ 。
- 点数边数混用：`vis[N]`、`dist[N]` 是点属性，`edge[M]` 是边属性，不要反着开。
- 多测清空：上一组 $N = 10$ ，下一组 $N = 5$ 时，只清空 $1 \sim 5$ 会留下 $6 \sim 10$ 的残边。
- 编号基准：输入是 $0 \sim N - 1$ 还是 $1 \sim N$ ，建图和循环必须一致。

2.1.2 图的特殊形态

- 不连通：题目没保证连通时，不能只从 1 号点 DFS。外层加 `for` 枚举所有未访问点。
- 重边：邻接矩阵存最短路时写 `g[u][v] = min(g[u][v], w)`。
- 自环：拓扑入度、无向图度数、连通性都可能受影响。必要时读入时 `if (u == v) continue`；。
- 退化图：链会爆递归栈，菊花图会卡某些 $O(N^2)$ 暴力。

2.1.3 DFS 与 BFS

- 无向树 DFS：必须传父亲 `dfs(u, fa)`，循环中跳过 `if (v == fa) continue`；。
- BFS 标记时机：入队时立刻 `vis[v] = true`，不要等出队才标记。
- 回溯恢复：找所有路径时，离开递归前记得恢复 `vis[u] = 0`。

2.1.4 最短路

- Dijkstra 去重：堆优化版本出队后写 `if (vis[u]) continue`；。

- 负权边：有负权时 Dijkstra 失效，改用 Bellman-Ford、SPFA 或重构模型。
- Floyd 初始化：先设为 `inf`，对角线 `dp[i][i] = 0`。
- Floyd 顺序：循环顺序必须是 k, i, j 。
- 松弛防溢出：写 `dist[u] != inf && dist[v] > dist[u] + w`。

2.1.5 拓扑排序

- 判环：拓扑结束后检查出队数量 `cnt == n`。否则图中有环。
- 字典序：要求字典序最小时，用 `priority_queue<int, vector<int>, greater<int>>`，不是普通队列。

2.1.6 LCA 与树形 DP

- 倍增维度：第二维按 $\log N$ 开足，循环边界不要少一层。
- 预处理顺序：倍增表通常外层枚举步长，内层枚举节点。
- 树形 DP 初始化：在 `dfs(u)` 开头清理并设置当前点的 DP 状态，避免父节点读到旧数据。
- 后序转移：必须先 `dfs(v)`，再用 `dp[v]` 更新 `dp[u]`。

图论救场法：一直 WA/RE 时，在测试数据里加孤立点、自环、重边、链和菊花图，很多隐藏越界和死循环会立刻出现。

2.2 动态规划

2.2.1 状态定义与初始化

- 求最大值初始化为 0：如果收益可能为负，答案可能是 -5 ，0 会把正确答案盖掉。最大值初始化为 $-\infty$ ，最小值初始化为 ∞ 。
- 幽灵状态转移：`dp[j] == inf` 时不要参与 `dp[i] = min(dp[i], dp[j] + cost)`。
- 状态边界歧义：“前 i 个元素的最优值”和“以第 i 个元素结尾的最优值”不是同一个状态，答案位置也不同。

2.2.2 遍历顺序

- 区间 DP：最外层枚举区间长度 len ，再枚举左端点 L ，推出右端点 R 。
- 0-1 背包：容量倒序遍历；完全背包容量正序遍历。
- 树形 DP：儿子的状态算完后，父亲才能转移。

2.2.3 状态转移遗漏

- 白手起家：LIS 每个元素都可以单独成为长度 1，初始化 $dp[i] = 1$ 。
- 容斥重复：计数 DP 有多个否定条件时，注意路径或方案被重复减掉。
- 滚动数组脏数据：每一轮开始先清空当前层；注意 $+=$ 和 $=$ 的区别。

2.2.4 边界与答案位置

- 负数下标：转移含 $dp[i-2]$ 时，循环不能从 $i=1$ 开始。
- 背包容量： $j - weight[i]$ 必须非负。
- 提取答案：状态是“以 i 结尾”时，答案常常是 $\max(dp[1\dots n])$ ；背包不要求装满时，可能是 $\max(dp[n][0\dots V])$ 。

DP 破局法：找一组最小样例，把 DP 表完整打印出来。手算一张表，对比第一处不同的格子，那里通常就是状态定义或转移方程的错误。

2.3 数据结构与技巧

2.3.1 并查集

- 假合并：不要写 $fa[u] = v$ 或 $fa[find(u)] = v$ ；正确写法是 $fa[find(u)] = find(v)$ 。
- 初始化：每组数据开头执行 $iota(fa + 1, fa + n + 1, 1)$ 。
- 统计连通块：统计 $find(i)$ 的种类，或统计根节点 $fa[i] == i$ 。

2.3.2 线段树

- Lazy Tag 混淆：区间加法的标记是累加，区间赋值才是覆盖。

- **Pushdown 不完整**：下传标记时，既要传 tag，也要同步更新子节点的 tree 值，最后清空父标记。
- **区间长度写错**：更新左儿子用原节点范围 $\text{mid} - 1 + 1$ ，不是查询边界 L。
- **越界返回**：区间最大值返回 $-\infty$ ，最小值返回 ∞ ，求和返回 0。

2.3.3 二分查找

- **死循环**：若分支会写 $l = \text{mid}$ ，则 mid 要上取整： $\text{mid} = (l + r + 1) / 2$ 。
- **溢出**：安全写法是 $\text{mid} = l + (r - l) / 2$ 。
- **浮点二分**：建议固定循环次数，例如 100 次，避免精度边界造成死循环。
- **check 污染**：每次 $\text{check}(\text{mid})$ 开头清空 vis、计数器和辅助数组。

2.3.4 双指针与滑动窗口

- **左指针越过右指针**：移动左指针时写 $\text{while } (l \leq r \ \&\& \ \text{sum} > K)$ 。
- **答案更新时机**：最长区间通常在修复合法后更新；最短区间通常在即将破坏条件前更新。
- **漏掉最后一段**：分段统计时给数组或字符串加哨兵，例如 $s += \text{'\#'};$ 。

2.3.5 位运算：优先级与集合操作

位运算的按位与、按位或、按位异或，优先级低于比较运算符。 $\text{if } (s \ \& \ 1 == 0)$ 会被解析成 $\text{if } (s \ \& \ (1 == 0))$ ，即 $s \ \& \ 0$ ，逻辑彻底错乱。

修正：位运算和比较混用时一律加括号： $\text{if } ((s \ \& \ 1) == 0)$ 。优先级从高到低：移位 $\ll$ 、比较 $< > ==$ 、按位与/或/异或、逻辑与/或——按位运算比较还低，所以几乎总要补括号。

取并集 vs 取交集（状压 / bitmask 高频混淆）：**按位或是并集**，越或越大；**按位与是交集**，越与越小。

- **并集 (union)**：合并集合 $A \mid B$ ；把元素 i 加入集合 $S \mid= (1 \ll i)$ 。
- **交集 (intersection)**：公共元素 $A \ \& \ B$ ；判断元素 i 是否存在 $\text{if } (S \gg i \ \& \ 1)$ 。
- **删除元素 / 差集**：删除元素 i 写 $S \ \&= \sim(1 \ll i)$ ；确定存在时翻转写 $S \ \hat{=} (1 \ll i)$ 。

- 常见 WA：想取并集却写成 $\&$ ，集合越并越空；想判断“同时拥有”却写成 $|$ 。

排错小动作：线段树出错就在 query 和 update 第一行打印当前节点区间；二分和双指针出错就手动模拟“答案在第一个位置、最后一个位置、完全无解”三种样例。

2.4 数学与精度

2.4.1 取模

- 减法取模： $\text{ans} = (A - B \% \text{MOD} + \text{MOD}) \% \text{MOD}$ 。
- 除法取模：组合数不能直接除，要用逆元。
- 临时溢出：连乘写成 $A * B \% \text{MOD} * C \% \text{MOD}$ 。

2.4.2 组合计数

- 容斥错觉：“至少包含 A 或 B”不能简单相加，否则同时包含 A 和 B 的方案会被算两次。
- 组合数边界：函数开头写 `if (m < 0 || m > n) return 0;`，并保证 `fact[0] = 1`。
- 有序与无序：球是否相同、人是否相同、是否允许空，都会改变公式。

2.4.3 数论

- LCM 溢出：写 $A / \text{gcd}(A, B) * B$ ，不要先乘后除。
- 分解质因数：循环结束后检查 `if (N > 1)`，剩下的部分就是一个质数。
- 负数整除：C++ 的负数除法向 0 取整；需要数学下取整时要手写处理或用 `floor`。

2.4.4 浮点精度

- 整数幂：不要用 `pow` 算整数幂，写快速幂。
- 完全平方数：写 `long long r = round(sqrt(N)); if (r * r == N) ...`。
- double 比较：相等用 `abs(A - B) < 1e-9`，大于用 `A - B > 1e-9`。
- 分式比较：优先交叉相乘，但要确认分母同号，并注意乘积是否需要 `__int128`。

3 读题错误

3.1 数值范围没看清

3.1.1 变量可以是负数或 0 吗

题面约束可能是 $-10^9 \leq a_i \leq 10^9$ ，但代码默认所有数都是正数。

- 最大值初始化为 0 会错。
- 乘法中两个负数会变成大正数，可能破坏单调性。
- 双指针依赖“全为非负数”的性质时，负数会让窗口单调性失效。

3.2 核心对象定义没看清

3.2.1 Permutation 还是 Array

题面写的是 `permutation`，意味着元素互不相同，且通常值域是 $1 \sim N$ 。如果当成普通数组，会浪费或错过关键性质。

修正：看到 `permutation` 时，优先考虑位置映射 `pos[a[i]] = i`、置换环、值域与下标互映射。

3.2.2 图是不是简单图

题面如果没有写 “The graph contains no self-loops and multiple edges”，后台很可能有重边和自环。

修正：建图前确认是否需要过滤自环、保留最小重边、或者让算法天然支持重边。

3.2.3 有向还是无向

“A 和 B 是朋友”可能是无向关系；“A 关注 B”通常是有向关系。背景语言会影响建边方向。

修正：看清 Directed / Undirected，以及样例解释中边是否双向。

3.3 操作规则读错

3.3.1 同时发生还是依次发生

题面写 *simultaneously*，但代码在双层循环里直接修改原数组，就会让新状态影响本轮后续判断。

修正：使用新数组、临时列表或队列记录所有变化，本轮判断结束后统一修改。

3.3.2 代价是单次还是全局

“删除一个元素花费 X ”和“执行一次删除操作总花费 X ，可删多个元素”是两种完全不同的 DP。

修正：看到 *cost* 时，逐字确认它乘的是删除数量、操作次数、区间长度，还是固定一次。

3.3.3 Any 还是 All

All 要求所有对象满足；Any / At least one 只要求存在一个。读反后算法方向会完全错。

修正：把题面量词圈出来：all、any、exactly、at most、at least、for every、there exists。

3.4 坐标系与格式

3.4.1 行列坐标与笛卡尔坐标

输入 (X, Y) 常常表示横坐标和纵坐标，但 C++ 数组是 `grid[row][col]`。如果直接写 `grid[x][y]`，地图可能被转置；长宽不同还会越界。

修正：看图示和变量含义：输入到底是 (R, C) 还是 (x, y) 。必要时读入后写 `grid[y][x]`。

3.4.2 矩阵的顺时针与逆时针旋转

把 n 行 m 列的矩阵旋转 90° 时，行列下标、新矩阵尺寸、顺逆方向最容易搞反。旋转 90° 后尺寸一定从 $n \times m$ 变成 $m \times n$ 。

设原矩阵 a 为 n 行 m 列（下标从 0 起），旋转后矩阵记为 b 。

- **顺时针 90°** ： $b[j][n-1-i] = a[i][j]$ ，即 $b[i][j] = a[n-1-j][i]$ 。口诀：先转置，再把每一行左右翻转。
- **逆时针 90°** ： $b[m-1-j][i] = a[i][j]$ ，即 $b[i][j] = a[j][m-1-i]$ 。口诀：先转置，再把行的顺序上下翻转。
- **旋转 180°** ： $b[n-1-i][m-1-j] = a[i][j]$ ，尺寸不变。

修正：记不住就画一个 2×3 小矩阵，盯住 `a[0][0]` 转到哪个角，方向立刻清楚。非方阵必须用新数组并交换行列尺寸，不能原地旋转。

3.4.3 输入顺序

不是所有题都先给 N 再给 M 。有些题是 `cin >> M >> N;`，先列后行，或者先边后点。

修正：读题时给变量旁边写中文注释：点数、边数、行数、列数、横坐标、纵坐标。

3.4.4 无解输出

凭经验输出 `-1`，但题目可能要求输出 `0`、空行或 `Impossible`。

修正：提交前回到题面 `Output` 段落，专门确认无解、多解、大小写和空行规则。

A 常用 C++ 板子

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int inf = 2e18;
// const int inf = 0x3f3f3f3f;
const int N = 1e6 + 10;

void solve() {
    int n = 1;
    cin >> n;
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```